

Lab 0-F26 Zephyr

ESE 5180: Wireless, Security, & Scaling

Gradescope PDF Submission Deadline: Sep 16, 2026 11:59 PM

Teaching Team Checkoff Deadline: Sep 22, 2026

An **individual** submission to be submitted on GitHub & Gradescope.

GitHub Repository Template: <https://github.com/e5180/lab0-zephyr-skeleton>

You must create a private repository within the **e5180** GitHub organization.

Late-day requests: <https://forms.gle/YkvTbsuNCR2wdZLN8>

Overview

In this lab, you will set up the Zephyr RTOS build system on a few different microcontroller boards, write and build code using the West tool, and flash/debug your firmware directly on hardware. You will also learn the role of Kconfig and the Device Tree in configuring drivers and peripherals such as I²C or SPI sensors. Finally, you will integrate the BME280 I²C humidity and temperature module.

All the required questions are highlighted in green.

Important notes are highlighted in yellow.

Required Equipment

- Textbook
 - [Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API](#) by Andrew Elias
- Hardware
 - Personal laptop computer (Windows, Mac OS, Linux)
 - [XIAO nRF52840\(sense\)](#) & USB C cable
 - [STM32 Nucleo F411RE](#) & mini USB cable (or [NUCLEO-WL55JC](#))
 - [Nordic nRF7002 DK](#) & micro USB cable
 - [Adafruit BME280 I2C or SPI Temperature Humidity Pressure Sensor](#)
 - [STEMMA QT / Qwiic JST SH 4-pin Cable with Premium Female Sockets](#)
 - [STEMMA QT / Qwiic JST SH 4-pin to Premium Male Headers Cable](#)
- Software
 - [Visual Studio Code \(VS Code\)](#)
 - [nRF Connect Desktop App](#)

1. Hello (Vanilla) Zephyr

Companion Reading:

[Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API](#) by Andrew Eliasz

Chapter 3. Zephyr RTOS Application Development Environments and Zephyr Application Building Principles

References:

[Getting Started Guide — Zephyr Project Documentation](#)

One main reason for using Zephyr is its portability. As you'll see, there is a lot of abstraction in Zephyr, as well as supporting code, that allows it to build for (currently) over 1000x MCU boards! Let's see how portable a simple "blink" code can be by building for three separate MCU boards: a XIAO Sense, STM32 Nucleo F411RE, and a Nordic nRF7002 DK.

Follow Zephyr's documentation for setup. We're calling it "vanilla" Zephyr here because it has no embellishments or changes (foreshadowing here for Nordic...). You could also call it "stock" Zephyr.

[Getting Started Guide — Zephyr Project Documentation](#)

Don't miss this note from the Zephyr install page!

Python 3.12 is strongly recommended. Using a newer Python release may fail on some systems, for example when installing the required packages on Windows.

Installation size can be somewhat big - looks like it's about 20GB on my Windows machine. Make sure you have enough space!

When everything has been installed, you'll get to a section entitled [Build the Blinky Sample](#). Using the [Supported Boards and Shields](#) page, determine the build name for the XIAO Sense, STM32 Nucleo F411RE (the STM32 WL55JC is a suitable replacement here), and a Nordic nRF7002 DK. In turn, build the blinky example and flash it to each one of the MCU boards.

(1.1) Create a video showing blinky on all 3x MCU boards.

- In the video, show your terminal history of compiling and flashing for all 3x MCU boards.
- In the video, show each MCU board flashing its LED while running the blinky code.
- Name the video file: **f26_lab0_1.1_pennkey**
- Submit the video to [this Google Form](#).
 - The Google Form has submissions for each video question in this lab. If you complete video 1.1, you can submit it immediately, then return to the form later to submit video 1.2. Or, you may submit all videos at once.

- The Google form submissions have a size limit. We don't need 4K quality - just enough to see what's required of the assignment.

2. Hello (Nordic) Zephyr

Companion Reading:

[Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API](#) by Andrew Elias

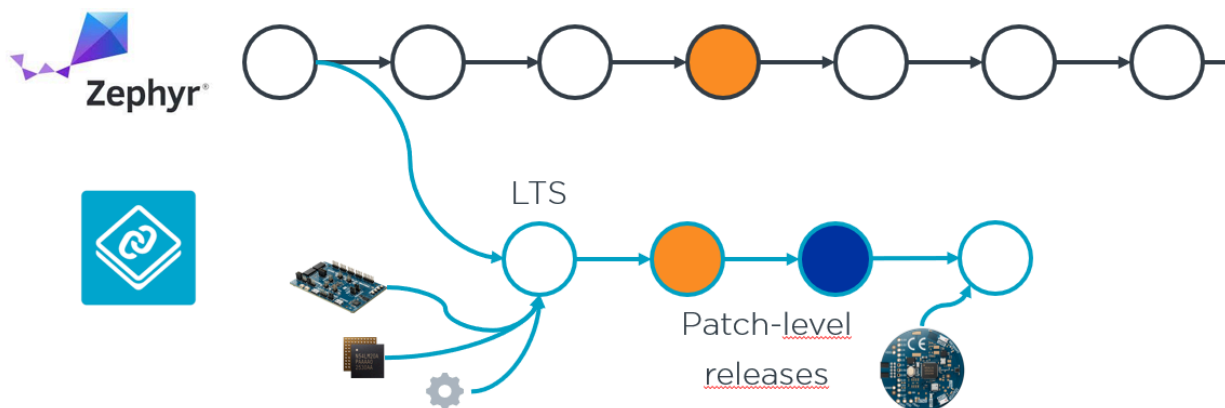
Chapter 3. Zephyr RTOS Application Development Environments and Zephyr Application Building Principles

References:

[Installing the nRF Connect SDK](#)

[nRF Connect SDK Fundamentals - Nordic Developer Academy](#)

Zephyr is an open-source modular RTOS used widely in embedded systems. Nordic integrates Zephyr as the core of their nRF Connect SDK (NCS) - this means it's not stock. Here's an image visually describing how Nordic's Long Term Support (LTS) releases will follow the Zephyr mainstream release. ([More details here](#))



2.1 Development Environment Setup

The nRF Connect extension gives you access to a host of tools with device tree editing, RTT terminal, Kconfig editing, Cmake, compilers, and linkers, some of which we will explore in our lab, but we recommend you experiment with all of them at your convenience.

Open the free [nRF Connect SDK Fundamentals](#) course.

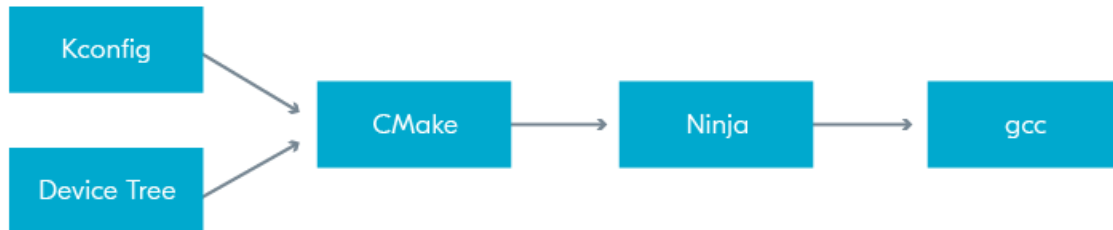
Follow [Exercise 1 - Installing nRF Connect SDK and VS Code](#).

Use **v3.4.x** for your projects this semester - it is a Long Term Support (LTS) release (more information here: [nRF Connect SDK gets Long-term support with version 3.4.0 - Blogs](#)).

2.2 Flashing an Example

Follow [Exercise 2 – Build and flash your first nRF Connect SDK application](#) - and read the brief explainer below. You'll learn how to use the Nordic development process - which will be integral to our projects for the remainder of the semester.

Here's an overview of the SDK tools:



Ensure you select the board target, in our case nrf7002dk - **nrf7002dk/nrf5340/cpuapp/ns**.

Let's break down what this board target means:

nrf7002dk/nrf5340/cpuapp/ns

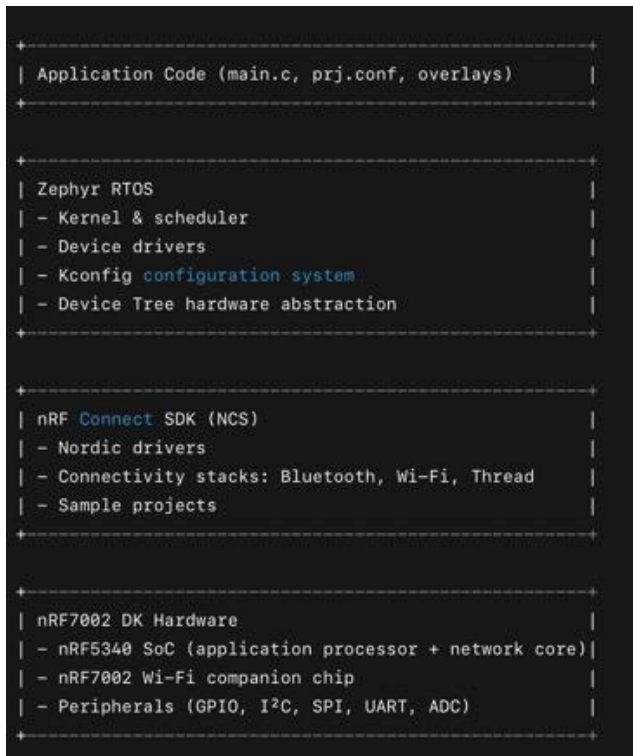
- **nrf7002dk**: our Nordic nRF7002 development kit (dk). Note: the nRF7002 is a companion IC that cannot run application firmware.
- **nrf5340**: This is a BLE-enabled MCU that can run application code - the host processor. It exists on the dev kit to communicate with the companion nRF7002 Wi-Fi IC.
- **cpuapp**: The nRF5340 has two cores - the cpunet for running the BLE network core and the cpuapp for running the application code. We're loading an application, not a new wireless networking image.
- **ns**: This loads the firmware in the non-secure section of the nRF5340, meaning it has limited access and permissions. The nRF5340 supports ARM's TrustZone technology.

Note, if you want to debug, set Optimised for debugging -Og

Internally, the nRF Connect SDK code is organized into four main repositories:

- nrf – Applications, samples, connectivity protocols (Nordic)
- nrfxlib – Common libraries and stacks (Nordic)
- Zephyr – RTOS & Board configurations (open source)
- MCUBoot – Secure Bootloader (open source)

Overview of your codebase:



We also recommend that you:

- Check the format of the build image/executable
- Check the internal commands VS Code passes to the terminal to build
- Check the internal command VS Code passes to the terminal to flash

How does this compare to the “vanilla” Zephyr setup?

Zephyr applications are organized with CMake at their core. A typical project layout includes:

- src/ – application source code
- prj.conf – Kconfig settings
- boards/ – optional devicetree overlays

Navigate to src/ within your application directory to find your **main.c**. Take the Blinky example in Zephyr and change the following: Switch the LED blinking time to 2 seconds, build and flash again.

Note that the SDK added the following parts to your project. You should know what each of these parts does in your application. Consult the project’s README.rst for additional information

- boards and build directories

- CMakeLists
- Proj.conf
- .yaml

(2.1) Commit your Zephyr application to your GitHub repository

(2.2) Create a video showing the change in blinky behavior on the nRF7002DK

- In the video, show your terminal history within VS Code
- In the video, show the nRF7002DK board blinking at its original rate, then change the blinking time to 2 seconds, flash, and show the execution.
- Name the video file: **f26_lab0_2.1_pennkey**
- Submit the video to [this Google Form](#).

3. Building with West

Companion Reading:

[Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API](#) by Andrew Elias

Chapter 4. Zephyr RTOS Multithreading

[West \(Zephyr's meta-tool\)](#) - Zephyr Documentation

`west` is Zephyr's meta-tool. It solves two problems: managing Zephyr's multi-repo workspace (via a manifest) and providing convenient extension commands for common workflows—**build**, **flash**, **debug**, etc. Zephyr's docs describe `west` as a pluggable multi-repo manager with build/flash/debug conveniences baked in.

You have used the direct commands through the GUI from nRF Connect. Now, let's explore what each of these commands does internally.

West is Zephyr's meta-tool that wraps CMake/Ninja. We'll use some of the key commands in this section. Here are the core four commands to know.

1. `west init`
 - a. Ex: `west init -m https://github.com/nrfconnect/sdk-nrf`
2. `west update`
3. `west build`
 - a. Ex: `west build --build-dir your_app_dir/build your_app_dir --pristine --board nrf7002dk/nrf5340/cpuapp/ns --sysbuild -DBOARD_ROOT=your_app_dir`
4. `west flash`

- a. Ex: `west flash -d your_directory/build --dev-id 1050746992 --erase`

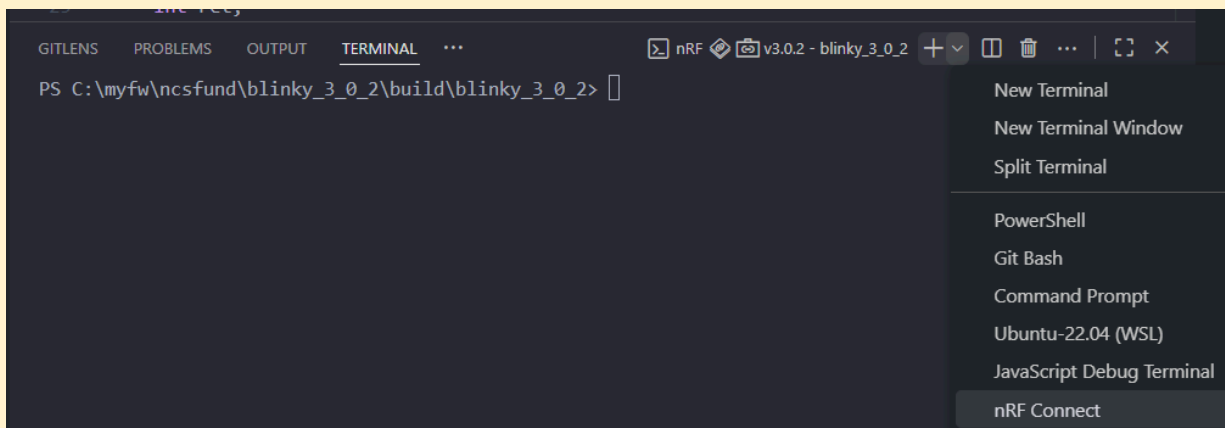
Think: Why does Zephyr wrap CMake with West?

(3.1) Build and Flash using only west commands and not the GUI. Show the terminal prints by embedding a screenshot in your README.md.

West not recognized? An easy fix is to use the nRF Connect terminal instead of a generic terminal on your machine.

The nRF Connect terminal correctly sets up the environment so you may run west commands.

In the Terminal window, click the drop-down arrow and find **nRF Connect**.



Challenge Questions:

1. Give a 1-line explanation of west init, update, build, and flash.
2. In your build and flash commands, include a list of each positional argument/flag you pass to build and what they mean.

Hint: Use `-h` to learn about each of the commands.

At this point, you have learned that there are two main workflows:

- Using the nRF Connect Programmer app
- Using West directly: west flash, west debug

Now, we'll dig deeper into the configuration of the Zephyr application.

4. Kconfig

Companion Reading:

[Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API](#) by Andrew Elias

Chapter 4. Zephyr RTOS Multithreading

Kconfig is Zephyr's configuration system, enabling control over OS-level features, drivers, and application-specific settings. It uses a textual configuration language to define symbols (options) and their dependencies, allowing developers to tailor the Zephyr build to their specific hardware and software requirements. Kconfig controls the software configuration of a Zephyr project, whereas Device Tree handles the hardware configuration.

4.1 Kconfig Implementation Details

1. **prj.conf File:** This is the primary file where Kconfig settings for your application are typically stored. Each line represents a Kconfig symbol and its value.
For example:
 - `CONFIG_LOG=y`: Enables the logging subsystem.
 - `CONFIG_LOG_DEFAULT_LEVEL=3`: Sets the default log level to 'info'.
2. **Interactive Menu (`menuconfig`):** While you can directly edit `prj.conf`, Zephyr provides an interactive menu-driven configuration tool. You can access it by running `west build -t menuconfig` in your project directory. This tool allows you to navigate through a hierarchical menu of all available Kconfig options, enabling or disabling features and setting their values. It also helps in understanding dependencies and conflicts between different options.
3. **Relationship between `prj.conf` and `menuconfig`:**
 - `prj.conf` acts as an overlay or initial set of configurations. When you run `west build`, the Kconfig system processes `prj.conf` along with default configurations and board-specific settings.
 - `menuconfig` provides a visual and interactive way to view and modify these settings. Any changes made in `menuconfig` are typically saved back into your project's configuration files.

(often in a hidden `.config` file within the build directory), which then influence the subsequent builds.

4. **Verification of Settings:** After building your project, the Kconfig system generates a `.config` file in your build directory (e.g., `build/zephyr/.config`). This file contains the final resolved values for all Kconfig symbols, taking into account `prj.conf`, default settings, and any board-specific overrides. You can inspect this file to verify that the symbols defined in your `prj.conf` are correctly set and that there are no unexpected conflicts or omissions. This verification is crucial to ensure your application is built with the intended features and driver support.
5. **Integration with Build System:** Kconfig is tightly integrated with Zephyr's CMake-based build system. The Kconfig options determine which source files are compiled, which drivers are included, and how various subsystems are configured during the build process.

Think:

- What are the levels of Log statements?
- What is the difference between `prj.conf` and `menuconfig`?
- How do you check that the symbols in `prj.conf` are set after building? Why?

5. Device Tree (DT)

Companion Reading:

[Zephyr RTOS Embedded C Programming: Using Embedded RTOS POSIX API by Andrew Elias](#)

Chapter 9. Understanding and Working with the Device Tree in General and SPI and I2C in Particular

In Zephyr, **DeviceTree (DT)** is a hardware description system. Instead of hardcoding pins or drivers in C code, Zephyr uses `.dts` (DeviceTree Source) files that describe:

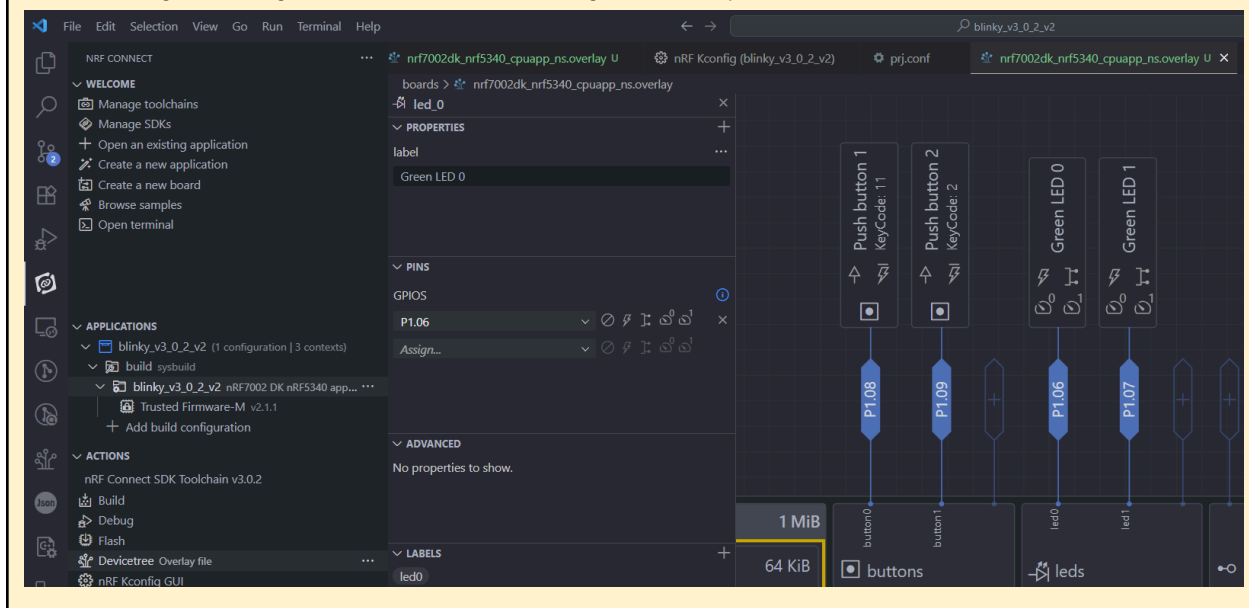
- **What hardware exists** (GPIO controllers, UARTs, LEDs, sensors, etc.).
- **How it is wired** (e.g., LED0 is on GPIO0 pin 13, active low).
- **Which devices should be enabled** (`status = "okay"` vs `"disabled"`).

At build time, Zephyr compiles `.dts` files + your app's overlays (`.overlay`) into C macros. Drivers and your code use those macros to bind to devices without knowing low-level details.

(5.1) Switch out the blinking to **LED2**. Create an overlay file with your own alias named **LED5180** that links to **LED2** on the nRF7002 DK. Ensure you only call this alias in the `main.c` program. Commit your code to this [GitHub repository](#).

Ensure that your overlay file is created in `app_dir/boards/` and has the same name as your board. Look this up in the target build configuration. Swap the backslashes for underscores.

Note: Take a look at the **Devicetree** overlay file within the **Actions** pane.
You can navigate through the entire hardware configuration for your board!



Let's explore a bit:

(5.2) Poll for a button press and switch the LED state.

Commit these application changes to your GitHub repository.

To accomplish this:

- Look up how a button is configured in your board [Hint: check the dts]
- Realise how it is similar to an LED, but is an input GPIO
- Use the same GPIO commands present in your example to configure the button press

(5.3) Create a new alias for your button and call your alias within your main.c.

Commit these application changes to your GitHub repository.

Think: Why create an overlay if you can edit the dts?

6. Printing vs. Logging

Resources

[CMake and Kconfig tricks for common code in Zephyr apps - The Goliath Developer Blog](#)

[Nordic Application Development](#)

[Debugging Zephyr for Beginners: printk\(\) and the Logging Subsystem - The Goliath Developer Blog](#)

In this section, you will implement a simple **sum function** in two different ways: once using **printk()** and once using the **Logger module**. The active implementation will be selected at build time using Kconfig options (`CONFIG_SUM_PRINT` and `CONFIG_SUM_LOG`). This assignment will help you practice console output, explore the differences between `printk` and the Logger, and understand how Kconfig choices can control application behavior.

To print basic messages to the console, we'll use the *printk()* function. Its syntax is very similar to the standard *printf()* in C: you can pass in a string literal or a format string along with variables you want to display. Unlike *printf()*, *printk()* is designed for embedded systems and supports only a simplified set of features, making it lightweight and efficient.

Enable console drivers

Make sure the console is enabled by setting **CONFIG_CONSOLE** in your application's configuration file.

Choose a console backend

Zephyr supports several backends, such as the UART console (`CONFIG_UART_CONSOLE`) and the RTT console (`CONFIG_RTT_CONSOLE`). For this lab, we'll use the **UART console**, since its output can be easily viewed in a serial terminal (for example, the built-in terminal in VS Code). On most boards, UART is already set as the default.

6.1 Logging

While `printk()` is useful for simple output, the **Logger module** is the recommended way to send messages to the console. Unlike `printk()`, which blocks until every byte of the message has been sent, the Logger offers more efficient and flexible logging. It supports both **in-place** and **deferred** logging and comes with several advanced features, including:

- Support for multiple backends
- Compile-time filtering at the module level
- Runtime filtering that can be set independently for each backend
- Timestamping with a user-provided function
- A dedicated API for dumping data
- Optional colored output in logs

- Integration with `printk()` (printk messages can be redirected through the Logger)

You can read the full list of features in the [Logging documentation](#).

Logs can also be compiled in, but filtered at run time using a dedicated API. The run-time filtering is independent for each backend and each module. There are four severity levels available in the system that can filter the warnings appropriately.

Your directory should look like this:

```
<app-root>/
├── src/
│   └── main.c
├── sum_printk/
│   ├── sum_printk.c
│   └── sum_printk.h
├── sum_log/
│   ├── sum_log.c
│   └── sum_log.h
├── prj.conf
├── Kconfig      <-- defines CONFIG_SUM_PRINT / CONFIG_SUM_LOG
├── testcase.yaml
├── CMakeLists.txt  <-- minimal app CMake
└── tests <-- for future sections
```

Instructions

1. Create the sum function

- Write a function that takes two integers, computes their sum, and returns the result.
- In `main.c`, call the sum function with two test values and display the result.

2. Two debugging implementations

- In `sum_printk/`, implement the version that uses `printk()` to show the result.
- In `sum_log/`, implement the version that uses the Logger to print logs of different severities and perform a hexdump of the input values to the function.

3. Kconfig setup

- Create a Kconfig file for the sum exercise (or use the same Kconfig specified in directory structure)
 - Remember to add `source "Kconfig.zephyr"` into your Kconfig, why is this necessary?
- Define a **choice** with two mutually exclusive options:
 - `CONFIG_SUM_PRINT` → builds the `printk()` implementation.
 - `CONFIG_SUM_LOG` → builds the Logger implementation.
- Default to `CONFIG_SUM_PRINT`.
- Hints:

- Review how Kconfig choice works.
- Use `#ifdef CONFIG_SUM_PRINT / #ifdef CONFIG_SUM_LOG` in `main.c` to select the right header and function call.

4. **prj.conf requirements**

- Enable the **console** and **UART** so that `printk()` can output.
- Enable the **Logger** when using the log implementation.
- Set the default log level to a sensible value (e.g., `INFO`).

5. **CMakeLists.txt**

- Minimal app file that builds the `src/` folder and conditionally includes `sum_printk/` or `sum_log/` depending on which option is active.
- Here's a good resource for the conditional including directories. Take a look at how `incore` semi implements conditional additions for files and folders:
[cmake/extensions.cmake · chromiteM · incoresemi / os_ports / zephyr · GitLab](https://github.com/chromiteM/semibuild/blob/master/cmake/extensions.cmake)
- Guard compilation with Kconfig so that only the selected module is built.

6. **Testing**

- Build and flash with `CONFIG_SUM_PRINT=y` → check that the result prints using `printk()`.
- Rebuild with `CONFIG_SUM_LOG=y` → check that the result prints using the Logger, with info-level messages and a hexdump of inputs.
- Capture output using a serial terminal (e.g., VS Code's built-in).

(6.1) Take screenshots of console output for **both builds**:

- `CONFIG_SUM_PRINT=y` → result printed with `printk()`.
- `CONFIG_SUM_LOG=y` → result printed with the Logger (include hexdump).

(6.2) Make a short video showing hexdump, log, and `printk`.

- Name the video file: **f26_lab0_6.2_pennkey**
- Submit the video to [this Google Form](#).

(6.3) Commit your updated Zephyr application to your GitHub repository

Think:

- What is the main difference between `printk()` and the Logger in terms of performance and flexibility?
- Additionally, why is sourcing Zephyr necessary in Kconfig?
- Why might deferred logging (Logger) be more suitable for embedded systems compared to `printk()`?

7. Ztest for Unit Testing

Resources

[Test Framework — Zephyr Project Documentation](#)

[Test Runner \(Twister\) — Zephyr Project Documentation](#)

[ZTest Doc](#)

[Beyond the Getting Started Guide — Zephyr Project Documentation](#)

Zephyr provides the Ztest framework for unit testing drivers and subsystems. You can write small tests and run them via west build/test integration. Let's try to add a test framework for our sum log tests.

A quick primer on how the [test framework](#) works in Zephyr:

- Zephyr uses [Board Simulation](#) / [Twister](#) to run tests
- You will be creating an additional test folder with the following:

```
<app-root>/
└─ LAB_1/
   └─ boards/
      └─ nrf7002dk_nrf5340_cpuapp_ns.overlay      # (optional board overlay)
   └─ src/
      └─ main.c                                  # main source for lab exercise
   └─ sum_log/
      └─ sum_log.c
      └─ sum_log.h
   └─ sum_printk/
      └─ sum_printk.c
      └─ sum_printk.h
   └─ tests/
      └─ SUM_UNIT_TEST/
         └─ build/                               # build output (auto-generated)
         └─ src/
            └─ sum_test.c                         # ztest-based unit test
         └─ CMakeLists.txt                       # minimal CMake for test app
         └─ Kconfig                             # feature configuration (printk/log)
         └─ prj.conf                            # app configuration
         └─ testcase.yaml                       # Zephyr test configuration
   └─ Kconfig                                   # top-level configuration
   └─ prj.conf                                 # main app configuration
   └─ CMakeLists.txt                           # top-level CMake project
   └─ .gitignore
   └─ readme.md                                # assignment guide
```

- **src/main.c** – holds the actual test cases and suite registration.
- **prj.conf** – same file you've already seen for applications, but here we add test-specific options (like enabling ztest or logging).
- **testcase.yaml** – tells Twister how to discover and run the test (name, board, tags, harness).

We'll need to implement a barebones Ztest framework for this:

1. Update your prj.conf with the following:

```
CONFIG_ZTEST=y
CONFIG_ZTEST_ASSERT_VERBOSE=2
CONFIG_LOG=y
```

Think: What does each of these configuration options do?

2. Update your testcase.yaml

Ensure to include 'tests:' as it tells Twister to pick this as a test

We are giving an initial file contents for your understanding, explore what each options do

```
tests:
  sum_log.test:
    platform_allow: qemu_cortex_m3      # comment out 'platform_allow' and run twister
    without -p flag to auto-select available boards
    tags: sum_log unit
    harness: ztest
```

3. Update your src/main.c within your test directory.

Resource: This has an example to use the ZTest: [Test Framework](#)

```
#include <zephyr/ztest.h>
#include "sum_log.h"

ZTEST(sum_log_test_suite, test_sum_log_basic)
{
    // TO DO
    //call sum_log(...) and zassert_equal(...)/zasser_true(...)
}

ZTEST(sum_log_test_suite, test_sum_log_negative)
{
    //TO DO
}

ZTEST(sum_log_test_suite, test_sum_log_zero)
{
    // TO DO
}

ZTEST_SUITE(sum_log_test_suite, NULL, NULL, NULL, NULL, NULL);
```

Note that the spacing (2 spaces per level) is required by Twister.

4. Now that we've set up our test, let's run it either with Board Simulator or Twister:

A: run in test directory (SUM_UNIT_TEST)

```
west build -p always -b qemu_cortex_m3 . --no-sysbuild
```

B: run in root directory (LAB_1)

```
west twister -T <your_test_directory> -p qemu_cortex_m3 --inline-logs -v
or
west twister -T tests/SUM_UNIT_TEST --inline-logs -v
```

You would have noticed that we added `qemu_cortex_m3` to test on your host laptop. The idea is to test both on our laptop (a much more powerful device that can rapidly run these tests) and our development board (the final hardware with the most accurate execution representation).

On Windows, does Twister build but not run? Your QEMU might not be installed or added to the path. Try:

```
choco install qemu -y #to install qemu outside of the nRF/Zephyr SDK tools
setx QEMU_BIN_PATH "C:\Program Files\qemu"
$env:QEMU_BIN_PATH = 'C:\Program Files\qemu'
echo $env:QEMU_BIN_PATH #check that the environment variable is working
Test-Path "$($env:QEMU_BIN_PATH)\qemu-system-arm.exe"
```

If the path turns out “true”, try running your Twister tests again.

Let’s take this framework and go further.

(7.1) Implement the test case in “TO DO” to test your function. Commit your updated Zephyr application to your GitHub repository

Hint: In your yaml **platform_allow**, add your target: `nrf7002dk_nrf5340_cpuapp_ns`

(7.2) Take screenshots of the testing outputs (laptop) in the terminal.

Think:

- Where is the `main()` in your Ztest? How is it still running? Describe how this happens in 2-3 sentences.
- What’s the difference between using “west twister” vs “west build” for the Ztest and in what scenario you would prefer which? (make a table of difference for this).

8. Adding a Peripheral (BME280)

Resources

[Connecting an BME280 temperature sensor - Nordic Developer Academy](#)

In this section, we will be pulling out the I2C implementation directly from the device tree and configure it for our use with the [BME280 I2C Temperature, Humidity, & Pressure Sensor](#). The updated DT leverages from this [nordic article](#) which will be worth a read!

Enable the I2C driver in Kconfig

```
# Sensor framework + I2C
CONFIG_SENSOR=y
CONFIG_I2C=y
```

Create your own I2C node in the DT **Note:** Add this to your overlay that you have already been using.

```
&i2c1 {
    status = "okay";
    pinctrl-0 = <&i2c1_default>;
        pinctrl-1 = <&i2c1_sleep>;
    pinctrl-names = "default", "sleep";
    <your_alias>:<your_alias>@77{
        compatible = "i2c-device";
        status = "okay";
        reg = < 0x77 >;
    };
};
&pinctrl {
    /omit-if-no-ref/ i2c1_default: i2c1_default {
        group1 {
            psels = <NRF_PSEL(TWIM_SCL, 1, 14)>,
                <NRF_PSEL(TWIM_SDA, 1, 15)>;
        };
    };
    /omit-if-no-ref/ i2c1_sleep: i2c1_sleep {
        group1 {
            psels = <NRF_PSEL(TWIM_SCL, 1, 14)>,
                <NRF_PSEL(TWIM_SDA, 1, 15)>;
            low-power-enable;
        };
    };
}; };
```

Additional notes:

1. Access your I2C node in your source code with `DT_NODELABEL(<your alias>)`
2. You might also require `CONFIG_CBPRINTF_FP_SUPPORT=y` in your conf to support printing of floating point operations.
3. Function calls that might point you in the correct direction:
 - `I2C_DT_SPEC_GET`
 - `I2c_write_read_dt`
 - `I2c_write_dt`
 - `i2c_burst_read_dt`

Hardware connections to make between the nRF7002DK and the BME280. Note: this operates the BME280 at 1.8V power and logic. This works with the Adafruit BME board you have.

Sensor board	nRF7002 DK
VCC	VDD
GND	GND
SDA	Port P1.15
SCL	Port P1.14

BME280 Register Map - from the datasheet:

<https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>

Register Name	Address	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0	Reset state	
hum_lsb	0xFE	hum_lsb<7:0>								0x00	
hum_msb	0xFD	hum_msb<7:0>								0x80	
temp_xlsb	0xFC	temp_xlsb<7:4>				0	0	0	0	0x00	
temp_lsb	0xFB	temp_lsb<7:0>								0x00	
temp_msb	0xFA	temp_msb<7:0>								0x80	
press_xlsb	0xF9	press_xlsb<7:4>				0	0	0	0	0x00	
press_lsb	0xF8	press_lsb<7:0>								0x00	
press_msb	0xF7	press_msb<7:0>								0x80	
config	0xF5	t_sb[2:0]			filter[2:0]			spi3w_en[0]		0x00	
ctrl_meas	0xF4	osrs_t[2:0]			osrs_p[2:0]			mode[1:0]		0x00	
status	0xF3					measuring[0]		im_update[0]		0x00	
ctrl_hum	0xF2					osrs_h[2:0]					0x00
calib26..calib41	0xE1...0xF0	calibration data								individual	
reset	0xE0	reset[7:0]								0x00	
id	0xD0	chip_id[7:0]								0x60	
calib00..calib25	0x88...0xA1	calibration data								individual	

(8.1) Print out the temperature. Take screenshots of your logging output. Commit your Zephyr application to your GitHub repository.

`Sensor_get_decoder` calls change, as we are no longer using the plug and play BME driver. Instead, you will have to configure and read directly from the temperature register.

For example, you will first set `ctrl_meas` register to active mode and then set oversampling to read:

Register 0xF4 "ctrl_meas"	Name	Description
Bit 7, 6, 5	osrs_t[2:0]	Controls oversampling of temperature data. See Table 24 for settings and chapter 3.4.3 for details.
Bit 4, 3, 2	osrs_p[2:0]	Controls oversampling of pressure data. See Table 23 for settings and chapter 3.4.2 for details.
Bit 1, 0	mode[1:0]	Controls the sensor mode of the device. See Table 25 for settings and chapter 3.3 for details.

And then read from `TEMP_MSB(0xFA)` for temperature and similarly `PRESS_MSB(0xFA)` for pressure

How can you pull out the temperature measurements?

- 1) I2C Write to `CTRL_MEAS`
- 2) I2C burst read to `TEMP_MSB`

Note: You will have to calibrate the sensor readings as well. Refer the sensor [datasheet](#).

(8.2) Create a Ztest to test if the device tree is set up and run sanity checks. Take and embed screenshots of your Ztest output for your README.md. Commit your Zephyr application to your GitHub repository.

Hint: This is not run on hardware, so you should consider using either a mocking function (returns values) or [FFF](#) to run these tests.

9. Teaching Team Checkoff

You must have submitted to Gradescope before requesting a Teaching Team checkoff!

Find a time before the lab submission to request a Teaching Team Checkoff. Office hours are available on the course website here: <https://ese5180.seas.upenn.edu/#officehours>

In the checkoff, you'll be expected to:

(9.1) Demonstrate and explain implementation details from the assignment: Sections [7. Ztest for Unit Testing](#) and [8. Adding a Peripheral \(BME280\)](#).

(9.2) Answer questions, without assistance, about the technology leveraged in this assignment and within the lectures:

-  L02-F26 Build Systems and Zephyr.pdf

-  L03-F26 Unit Testing TDD.pdf
-  L04-F26 CI and HIL.pdf

Hint: The **think** / **challenge** questions in purple are there for you to study and succeed! These will also be helpful for the in-class quiz on this subject matter.

Rubric

While the rubric attempts to capture all assignment details, points assigned may vary based on submission quality and teaching team review. Please read the assignment carefully to avoid missing details and losing points. Poor readability/formatting can lose you points on any assignment. See Gradescope for the rubric details.

For all questions, 0 points will be awarded if the submission is non-existent, poorly done, or doesn't compile (for firmware assignments).

Group assignments will have a feedback mechanism to help understand the distribution of work and collaboration. If there is a large disparity in work, grades may differ between students in the same group.

Appendix

Build Systems Resources

Read the following articles to inform your understanding of build systems and their use cases.

- [If They Come, How Will They Build It?](#)
- [The F5 Key Is Not a Build Process](#)
- [What Is a Build System? - Scott Dorman](#)

UPenn EE Master's alum Miles Osborne created a guide to a CMake-based build system here:

<https://embedded.seas.upenn.edu/Guides/build-systems/intro-to-build-systems/>

FAQ

nRF7002 /ns variant of board target does not work

Thanks to Cameron for his help with this. Here's the DevZone posts:

[Problems flashing non-secure apps to nrf7002dk - Nordic Q&A - Nordic DevZone - Nordic DevZone](#)
[NCS 3.4.0, compiling with ns option - Nordic Q&A - Nordic DevZone - Nordic DevZone](#)

Notable content:

From my understanding, this is caused by the migration from "partition manager" to devicetree partitions that came with v3.4.0. In NCS, by default, MCUboot is disabled but currently the "MCUboot partition" is still in the default partition of the /ns chips. So, currently, if you want to build for a /ns chip, you have 2 solutions:

Here's the overlay file Cameron generously provided for the exact nRF7002DK board we're using:

nrf7002dk_nrf5340_cpuapp_ns.overlay

```
// /delete-node/ &boot_partition;
// /delete-node/ &slot0_partition;
// /delete-node/ &slot1_partition;

// &flash0 {
//     partitions {
//         /* Primary image area, based at 0x0, sized to 960KB */
//         slot0_partition: partition@0 {
//             compatible = "zephyr,mapped-partition";
```

```

//          label = "image-0";
//          reg = <0x00000000 0xf0000>;
//          ranges = <0x0 0x0 0xf0000>;
//          #address-cells = <1>;
//          #size-cells = <1>;

//          slot0_s_partition: partition@0 {
//              compatible = "zephyr,mapped-partition";
//              label = "image-0-secure";
//              reg = <0x00000000 0x40000>;    /* 256 KB TF-M secure */
//          };

//          slot0_ns_partition: partition@40000 {
//              compatible = "zephyr,mapped-partition";
//              label = "image-0-nonsecure";
//              reg = <0x00040000 0xb0000>;    /* 704 KB non-secure app
*/
//          };
//      };
// };

```